PDK 4.HAFTA Notları

Arda Kumartaşlıoğlu

06-10-2025

1 Bölüm 3

- 3.1 Giriş
- 3.2 Söz Dizimini (Syntax) Tanımlamanın Genel Sorunu
- 3.3 Söz Dizimi (Syntax) Tanımlamanın Biçimsel Yöntemleri
- 3.4 Özellik Dil Bilgisi(Attiribute Grammars)
- 3.5 Programların Anlamlarını Açıklama: Dinamik Anlam Bilimi (Dynamic Semantics)

1.1 Giriş

Söz Dizimi (Syntax): Bir kodun doğru şekilde yazılış kuralı
Örneğin:

```
if (sayi\%2==0) # bu bir syntax örneğidir.
```

Semantik (Semantics): Kodun ne yaptığı, nasıl çalıştığı ve anlamı ile ilgilidir. Kod doğru olsa bile anlamsal olarak doğru olması gerekir.
Örneğin:

```
int x = 10;
int y = 0;
int z = x / y;
```

1.2 Lexeme:

Token veya en küçük sözcük birimidir. Derleyici veya yorumlayıcı, kaynak kodu okurken lexical analysis (lexing/tokenezleme) yapar ve kodu lexeme'lere ayırır. Örneğin:

```
x=10+5
```

Lexeme	Türü	
‘x‘	Identifier (değişken adı)	
‘=‘	Operator (atama)	
‘10‘	Constant/Literal	
‘+‘	Operator	
‘5‘	Constant/Literal	

1.3 Dillerin Biçimsel Tanımı

1.3.1 Tanıma Aracı(Recognizers)

Kodda geçerli bir yapı olup olmadığını kontrol eder.
Başka bir deyişle, bir dizi karakterin veya lexeme'in dil kurallarına uygun olup olmadığını "tanır".
Genellikle parser veya lexical analyzer kısmında kullanılır.

1.3.2 Üreteçler (Generators)

Kod veya çıktılar üretir; yani geçerli bir yapı oluşturur.
Genellikle derleyici veya dil tasarımı sırasında kullanılır.
Dilin kurallarına uygun doğru örnekler üretmek için kullanılır.

1.3.3 Context-Free Grammars

1950'lerin ortalarında Noam Chomsky tarafından geliştirilmiştir.
Doğal dillerin sözdizimini tanımlamak için kullanılan dil üreticileridir.
Bağlamdan bağımsız diller (context-free languages) olarak adlandırılan bir dil sınıfını tanımlayabilirsiniz.

1.3.4 Backus-Naur Form (1959)

Algol 58'in sözdizimini tanımlamak için John Backus tarafından icat edildi.
BNF, bağlamdan bağımsız dilbilgisine (context-free languages) eşdeğerdir.

1.3.5 LHS-RHS

LHS (Left-Hand Side) → Kuralın türetilen kısmıdır, yani hangi sembolün genişletileceği.
RHS (Right-Hand Side) → Kuralın üretim kısmıdır, yani LHS'nin neye dönüştüğü.

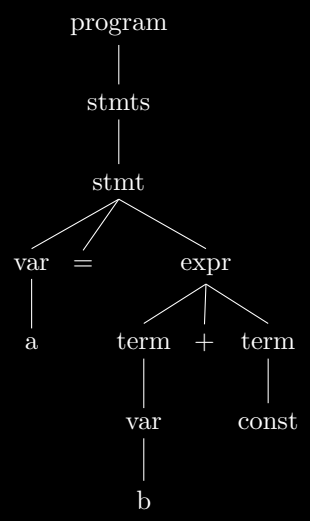
```
 E → E + T
 | Kısaltma | Açıklama                          |
 | -------- | --------------------------------- |
 | LHS      | ‘E‘ (sol taraf { türetilen ifade)    |
 | RHS      | ‘E + T‘ (sağ taraf { üretilen ifade) |

 | Kısaltma | Açılımı         | Anlamı                                          |
 | -------- | --------------- | ----------------------------------------------- |
 | **LHS**  | Left-Hand Side  | Kuralın sol tarafı (hangi sembol türetiliyor) |
 | **RHS**  | Right-Hand Side | Kuralın sağ tarafı (neye dönüştürülüyor)      |
```

Bir Grammer Örneği:

```
<program> -> <stmts>
<stmts>  -> <stmt> | <stmt> ; <stmts>
<stmt>  -> <var> = <expr>
<var>  -> a | b | c | d
<expr>  -> <term> + <term> | <term> - <term>
<term>  -> <var> | const
|-----------------------------------------------------------------|
<program> => <stmts> => <stmt>
=> <var> = <expr>
=> a = <expr>
=> a = <term> + <term>
=> a = <var> + <term>
=> a = b + <term>
=> a = b + const
```

1.3.6 Parse Tree



1.3.7 Ambiguity in Grammars (Gramerlerdeki Belirsizlikler)

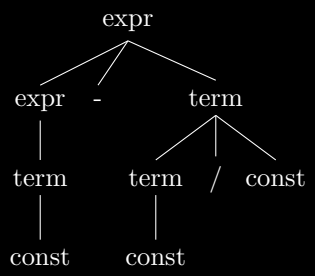
Bir gramer, aynı cümle (ya da ifade) için birden fazla farklı parse tree (türetim ağacı) oluşturabiliyorsa, bu gramer belirsiz (ambiguous) denir.
Yani dilin kuralları öyle tanımlanmıştır ki, hangi sırayla işlemlerin yapılacağı belli değildir.
Örneğin:

```
E → E + E | E * E | id
|-----------------------------------------|
Iki farklı yorum mümkün
| 1-(id + id) * id |

| 2-id + (id * id) |
```

İşte bu durumda gramer belirsizdir (ambiguous) çünkü aynı girdi için birden fazla parse tree üretilebiliyor.

1.3.8 Belirsiz Olmayan Bir İfadenin Grameri (An Unambiguous Expression Grammar)



1.4 Statik Anlam Bilimi (Static Semantics)

Program çalıştırılmadan önce **yani derleme aşamasında** kodun anlamsal (mantıksal) doğruluğunu kontrol eden kurallar bütünüdür.
Yani syntax (sözdizimi) doğru olabilir ama bazı şeyler mantıksal olarak yanlış olabilir. İşte bu tür hataları statik anlam bilimi yakalar.
Örneğin:

```
int x;
x = "merhaba";  // hata: tür uyuşmazlığı
```

Bu satır sözdizimsel olarak doğru (int x; doğru yazılmış), ama statik anlam hatası var çünkü "merhaba" bir string, x ise int.
Özetle:

Özellik	**Statik Semantics**	**Dinamik Semantics**
Ne zaman kontrol edilir	Derleme zamanında	Program çalışırken
Örnek hata	Tür uyuşmazlığı	Sıfıra bölme
Amaç	Kodun anlamını önceden doğrulamak	Kodun çalışma davranışını tanımlamak

1.4.1 Miras alınmış öznitelikler (Inherited Attributes)

Üst düğümden çocuklara doğru aktarılır. Yani bilgi yukarıdan aşağıya (top-down) akar.

1.4.2 Sentezlenmiş öznitelikler (Synthesized Attributes)

Çocuk düğümlerden ebeveyn düğüme doğru hesaplanır. Yani bilgi aşağıdan yukarıya (bottom-up) akar.

1.5 Anlam Bilimi (Semantics)

1.5.1 Operasyonel Semantics (Operational Semantics)

"Kod nasıl çalışır?" sorusuna cevap verir.
Programın her satırında ne olur, değerler nasıl değişir, bunları gösterir.
Süreç nasıl işler?
Kaynak kod → "ideal" bir makine diline çevrilir.
Bu makinenin nasıl davrandığı adım adım simüle edilir.

1.5.2 Denotasyonel Semantics (Denotational Semantics)

"Bu program hangi fonksiyonu temsil ediyor?" diye bakar.
Yani "çalıştırma" değil, "anlamını formülle ifade etme" yöntemi.
Scott ve Strachey tarafından 1970'lerde geliştirildi.
Özyinelemeli fonksiyon teorisine dayanır.
Matematiksel olarak en soyut (ama en sağlam) yöntemdir.

1.5.3 Aksiyomatik Anlam Bilimi (Axiomatic Semantics)

Bu yöntem, programın doğru çalışıp çalışmadığını matematiksel olarak ispatlamak için kullanılır.
Yani "program gerçekten istediğini yapıyor mu?" sorusuna cevap verir.
Aksiyomatik Semantics, "bir kodun anlamını" değil, "kodun doğruluğunu" tanımlar.
Yani "Eğer şu koşullar doğruysa, bu program sonunda şu sonucu üretir" der.

1. $P \Rightarrow I$ — Döngü başlamadan önce I doğru olmalıdır.
2. $\{I\}\ B\ \{I\}$ — Boolean koşulunun değerlendirilmesi I'yi değiştirmemelidir.
3. $\{I \wedge B\}\ S\ \{I\}$ — Döngü gövdesi çalıştığında I sabit kalmalıdır.
4. $(I \wedge \neg B) \Rightarrow Q$ — Eğer I doğru ve B yanlışsa, sonuç koşulu Q sağlanır.
5. *The loop terminates.* — Döngü sonlanmalıdır (bu genellikle en zor kısımdır).

Tür	**Ne yapar?**	**Odak noktası**	**Kullanım alanı**
Operasyonel	Kodu adım adım çalıştırır	Uygulama (nasıl çalışır)	Eğitim, dil kılavuzu
Denotasyonel	Kodu matematiksel fonksiyonlarla tanımlar	Anlam (ne yapar)	Teorik tanımlar
Aksiyomatik	Kodun doğruluğunu ispatlar	Doğruluk (doğru mu yapıyor)	Program doğrulama

Üç Semantik Yaklaşımın Karşılaştırması

Özet

- **BNF (Backus–Naur Form)** ve **Bağlamdan Bağımsız Gramerler (Context-Free Grammars)** eşdeğer meta dillerdir.
  - Programlama dillerinin **söz dizimini (syntax)** tanımlamak için oldukça uygundurlar.
- **Attribute Grammar**, bir dilin hem **söz dizimini (syntax)** hem de **anlam bilimini (semantics)** tanımlayabilen biçimsel bir yapıdır.
- **Semantics** (anlam bilimi) tanımlamanın üç temel yöntemi vardır:
  1. **Operasyonel Semantics** – Programın nasıl çalıştığını adım adım açıklar.
  2. **Aksiyomatik Semantics** – Programın doğruluğunu matematiksel olarak ispatlar.
  3. **Denotasyonel Semantics** – Programı matematiksel fonksiyonlarla tanımlar.